

Time : 40 minutes

Answer any three of the following questions

1. Discuss the evolution of microprocessors. 5
2. What are the basic differences between an accumulator based microprocessor and a general purpose microprocessor? Define RISC microprocessor. (2.5+2.5)
3. Differentiate between RISC and CISC type microprocessor. Discuss PUSH and POP operation of microprocessor. (2.5+2.5)
4. Discuss the FLAG register of 16 bit microprocessor. 5

MP Tutorial 1 solve

1. Discuss the evolution of microprocessors.

Definition of microprocessor

A **microprocessor** is the central unit of a computer system that executes instructions and processes data. It is a small, integrated circuit (IC) that functions as the **brain** of a computer or embedded system. Microprocessors perform arithmetic, logic, control, and input/output (I/O) operations as specified by a program.

Microprocessors have evolved significantly since their invention in the early 1970s. The evolution can be categorised into the following generations:

- **1st Generation (1971-1975):** Used **4-bit and 8-bit** processors like the **Intel 4004 (4-bit)** and **Intel 8080 (8-bit)**, **Intel 8085 (8-bit)**. These were **slow and had limited memory addressing capabilities**.
- **2nd Generation (1976-1980):** Featured **16-bit** microprocessors like **Intel 8086/8088**. These introduced **better performance, instruction pipelining, and memory addressing improvements**.
- **3rd Generation (1981-1990):** Introduced **32-bit** microprocessors such as **Intel 80386** and **Motorola 68020**, which **supported multitasking and virtual memory**.

- **4th Generation (1991-2000):** Used **superscalar** techniques with **Pentium processors (Intel Pentium, AMD K5)**. These **included cache memory and parallel processing features**.
- **5th Generation (2001-Present):** Modern processors like **Intel Core i-series, AMD Ryzen, and ARM processors** **use multi-core technology, high-speed caches, and power efficiency optimisations**.
- **Future Trends:** Microprocessors are moving towards AI-powered, quantum computing, and energy-efficient designs.

2. What are the basic differences between an accumulator-based microprocessor and a general-purpose microprocessor? Define RISC microprocessor.

Differences between an Accumulator-based and a General-Purpose Microprocessor:

Feature	Accumulator-Based Microprocessor	General-Purpose Microprocessor
Register Usage	Uses a single accumulator for operations.	Uses multiple general-purpose registers.
Instruction Set	Fewer instructions, simple architecture.	Many instruction sets, complex architecture.
Instruction execute	Instruction execute happens Sequentially	Instruction execute happens Parallel
Performance	Slower due to single register dependency.	Faster due to parallel execution.
Example	Intel 8085 (8 bit)	Intel 8086, ARM Cortex (16 bit)

Definition of RISC Microprocessor:

A **Reduced Instruction Set Computing (RISC)** microprocessor is a type of CPU that uses a small set of simple and highly optimised instructions rather than a

complex set of instructions for faster execution. These processors focus on **load/store architecture, fewer cycles per instruction, and pipelining** for better performance.

Examples: ARM Cortex, MIPS, PowerPC.

Key Features of RISC Microprocessors:

1. **Simple and Fixed-Length Instructions** – Each instruction performs a single operation, making execution faster.
2. **Load/Store Architecture** – Only **load (read from memory)** and **store (write to memory)** instructions access memory; all other operations use registers.
3. **Fewer Addressing Modes** – RISC processors support fewer and simpler addressing modes.
4. **Pipelining** – Allows multiple instructions to be executed simultaneously, improving performance.
5. **Large Number of Registers** – Minimizes memory access by keeping frequently used data in registers.

Examples of RISC Microprocessors

- **ARM Processors** (Used in smartphones and embedded systems)
- **PowerPC Processors** (Used in older Apple computers and gaming consoles)
- **MIPS Processors** (Used in networking and embedded applications)
- **RISC-V** (Open-source RISC architecture)

3. Differentiate between RISC and CISC type microprocessor. Discuss PUSH and POP operation of a microprocessor.

Differences between RISC and CISC Microprocessors:

Feature	RISC (Reduced Instruction Set Computing)	CISC (Complex Instruction Set Computing)
Instruction Complexity	Simple instructions.	Complex instructions.
Instruction Length	Fixed-length (e.g., 32-bit)	Variable-length (e.g., 8-bit to 64-bit)
Execution Speed	Faster due to fewer cycles per instruction.	Slower due to multiple cycles per instruction.
Memory Usage	Uses more RAM as instructions are simpler.	Requires less RAM as complex instructions perform multiple tasks.
Memory Access	Load/Store architecture (only load and store instructions access memory)	Direct memory access by most instructions
Addressing Modes	Fewer (simplified) addressing modes	Many addressing modes
Registers	Large number of general-purpose registers	Fewer registers, more memory operations
Pipelining	Highly optimised for pipeline and easier to implement due to uniform instructions.	Not optimised for pipeline and difficult to implement due to varying instruction lengths.
Power Consumption	Lower power consumption (used in embedded systems and mobile devices)	Higher power consumption (used in desktops and servers)
Examples	ARM, MIPS, PowerPC	Intel x86, AMD processors

Comparison Between RISC and CISC Microprocessors

- **RISC** focuses on **simplicity, efficiency, and speed**, making it ideal for **mobile devices, embedded systems, and modern CPUs**.
- **CISC** aims for **complexity and versatility**, making it suitable for **desktop computers, servers, and legacy systems**.

PUSH and POP Operations in Microprocessor:

- **PUSH Operation:**
- It stores (pushes) data onto the stack.
- The stack pointer (SP) **decreases** as data is stored.
- **Example:** PUSH AX, stores the contents of AX register onto the stack.
- **POP Operation:**
- It retrieves (pops) data from the stack.
- The stack pointer (SP) **increases** as data is retrieved.
- **Example:** POP BX, retrieves data from the stack into BX register

1. PUSH Operation:

The **PUSH** operation is used to store (push) data onto the **stack**. The stack follows a **Last In, First Out (LIFO)** principle. When data is pushed onto the stack, the **stack pointer (SP)** decreases.

PUSH Operation Steps:

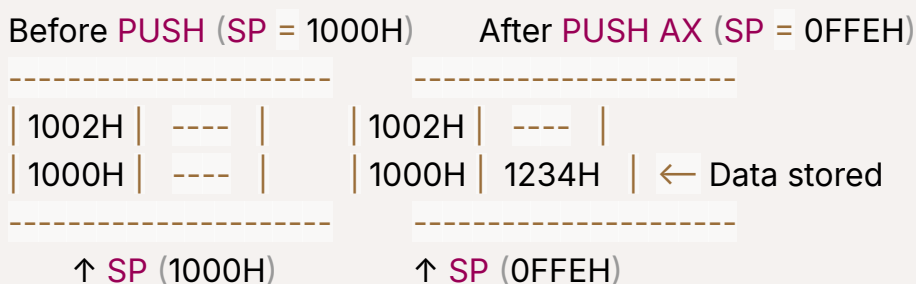
1. **Store the data** at the memory location pointed to by the SP.
2. **Decrement SP (Stack Pointer)** by 2 (for 16-bit processors like 8086).

Example of PUSH in 8086 Assembly:

```
MOV AX, 1234H ; Load AX register with value 1234H
PUSH AX       ; Push AX onto the stack
```

📌 This will store the value of AX (1234H) onto the stack and decrease SP by 2.

Diagram for PUSH Operation:



2. POP Operation:

The **POP** operation is used to retrieve (pop) data from the stack. When data is popped, the **stack pointer (SP)** increases.

Pop Operation Steps:

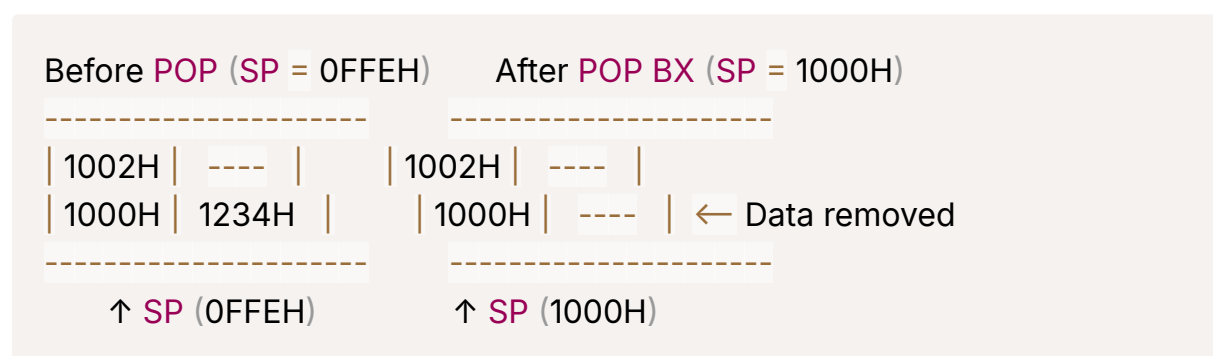
1. **Load the data** from the stack memory location pointed to by SP.
2. **Increment SP** by 2 (for 16-bit processors like 8086).

Example of POP in 8086 Assembly:

```
POP BX ; Pop the last pushed value into BX register
```

📌 This will retrieve the last pushed value (1234H in our example) into BX and increase SP by 2.

Diagram for POP Operation:



Key Differences Between PUSH and POP:

Operation	Action	Effect on Stack Pointer (SP)
PUSH	Stores data on stack	SP decreases
POP	Retrieves data from stack	SP increases

4. Discuss the FLAG register of a 16-bit microprocessor.

A **FLAG register** is a special register that holds the status of the processor after an operation. In a **16-bit microprocessor (Intel 8086)**, the **flag register** is **16 bits** and contains various flags that indicate the result of arithmetic and logical operations.

Main Flags in the 8086 Microprocessor:

1. **Carry Flag (CF):** Set when an arithmetic operation generates a carry beyond the MSB (Most Significant Bit).
2. **Parity Flag (PF):** Set if the number of 1s in the result is even.
3. **Auxiliary Carry Flag (AF):** Used in BCD operations, set if there is a carry from bit 3 to bit 4 bit.
4. **Zero Flag (ZF):** Set if the result of an operation is zero.
5. **Sign Flag (SF):** Set if the result is negative (MSB = 1).
6. **Trap Flag (TF):** Used for debugging, executes one instruction at a time.
7. **Interrupt Flag (IF):** Enables or disables interrupts.
8. **Direction Flag (DF):** Determines the direction for string operations.
9. **Overflow Flag (OF):** Set if an operation results in a value that is too large for the destination operand.

Flag Register Structure (16-bit)

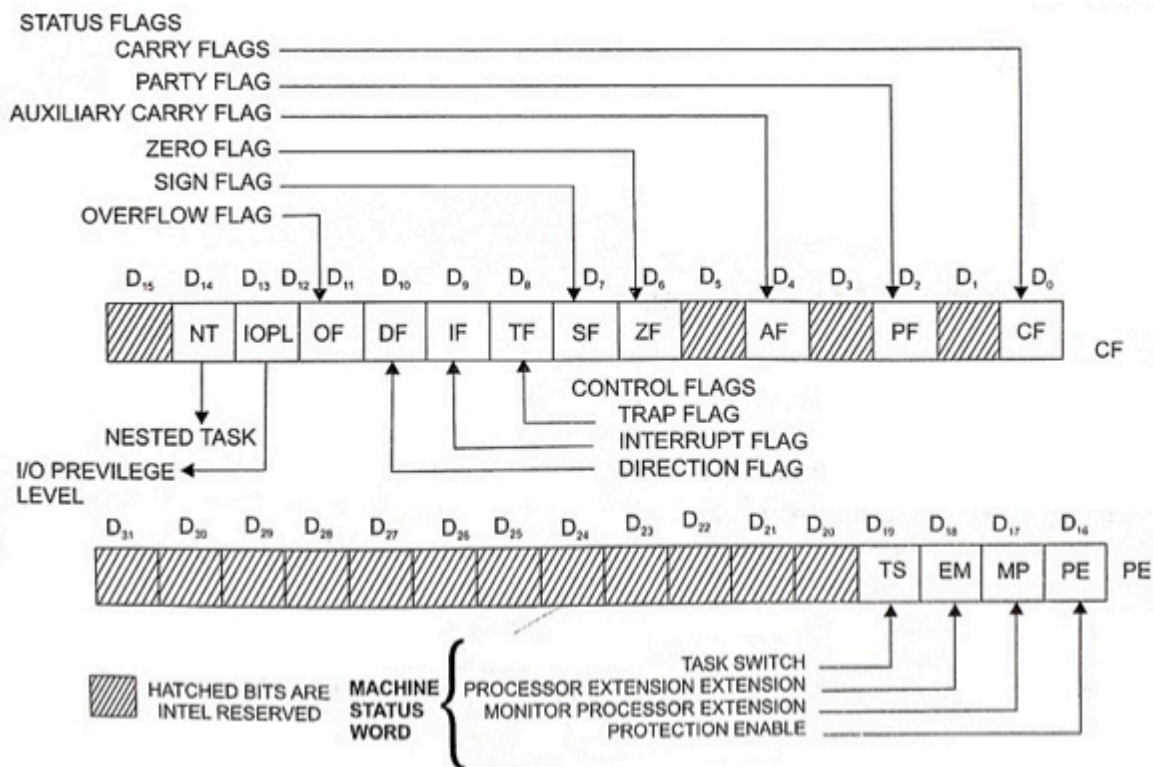


Fig. 11.17 80286 flag registers

| OF | DF | IF | TF | SF | ZF | 0 | AF | 0 | PF | 0 | CF |

The **flag register** is crucial for decision-making in assembly programming and execution control in microprocessors.

Conclusion:

These answers cover all the topics from your **CSE 357 (Microprocessors)** question paper. Let me know if you need further explanations or diagrams to clarify any concepts! 🚀